

Fronthaul Load Management in 5G C-RAN / O-RAN

Compression, rank-adaptive split selection and capacity-constrained allocation

B.Tech. Project — mid-term evaluation

September 2026

Evidence: Python/numpy simulation on 3GPP TR 38.901 TDL-A channels (no hardware, no field traces)

Legend used on every slide: [repo] present in the repository before this iteration [new] implemented and validated in this iteration [prelim] preliminary [future] planned

Why fronthaul load is the bottleneck

O-RAN split 7.2x carries frequency-domain I/Q per antenna, per subcarrier, per OFDM symbol

Uplink I/Q rate per radio unit (RU)

$$R_{\text{raw}} = 2 \cdot M \cdot N \cdot b \cdot (14 \text{ symbols} / 1 \text{ ms})$$

$M = 64$ antennas, $N = 1200$ subcarriers (15 kHz, 18 MHz), $b = 16$ bit

→ **2.46 Mbit per symbol = 34.4 Gbps per RU**

→ **8 RUs on one link: 275 Gbps**

Three levers, one budget

Compress the I/Q (how many bits, which structure to exploit)

Send fewer streams / move the functional split (rank-adaptive)

Share the link across RUs so the sum stays under capacity

What 'load management' means in this project

Per OFDM symbol and per RU, choose an encoder operating point (and stream count) such that

$\sum_k \text{rate}_k \leq \text{link capacity}$

while minimising the distortion of the transported matrices.

Not modelled (and not claimed):

MAC scheduling, HARQ, latency/jitter, hardware timing, real traces

Problem: several RUs, one fronthaul link, a fixed bit budget per symbol — decide bits, structure and streams per RU.

Starting point, audit and objectives for this iteration

What the repository actually contained on 17 Sep 2026 and what I set out to do

Component	Found in repository	Verified problem	Now
Experiment scripts Exp1-4, notebook, figures, report	[repo] present	import a package src/ that was never committed → nothing runnable; figures irreproducible	[new] src/ reconstructed, 42 tests
SVD baseline timing (Exp2)	[repo] figure	full N×N SVD timed (~1.7 s); economy SVD is 13 ms → '4-6x faster than SVD' unsupported	[fix] economy SVD
ACAFS split model (Exp3)	[repo] figure	'saving vs split 6' with split 6 as most expensive — inverted vs 3GPP TR 38.801; 'fast rank' curve = exact rank + random ±1	[fix] model + real estimator
NMSE vs SNR (Exp4)	[repo] figure	NMSE vs noisy Y saturates at −SNR; text claimed 'stable across SNR'	[fix] two references
CSEE input assumption	[repo] docs	delay sparsity only holds for unmodulated symbols; real data symbols whiten it	[new] data vs reference symbols
Shared-capacity allocation	absent	the 'load management' decision itself did not exist	[new] allocator + Exp5
Matrix-inversion benchmark	[repo] complete	NumPy engine used ReLU, model uses GELU; overstated docstring	[fix] + re-run

Priority order followed: (1) make it run and correct, (2) credible baselines, (3) one well-evaluated improvement — the allocator.

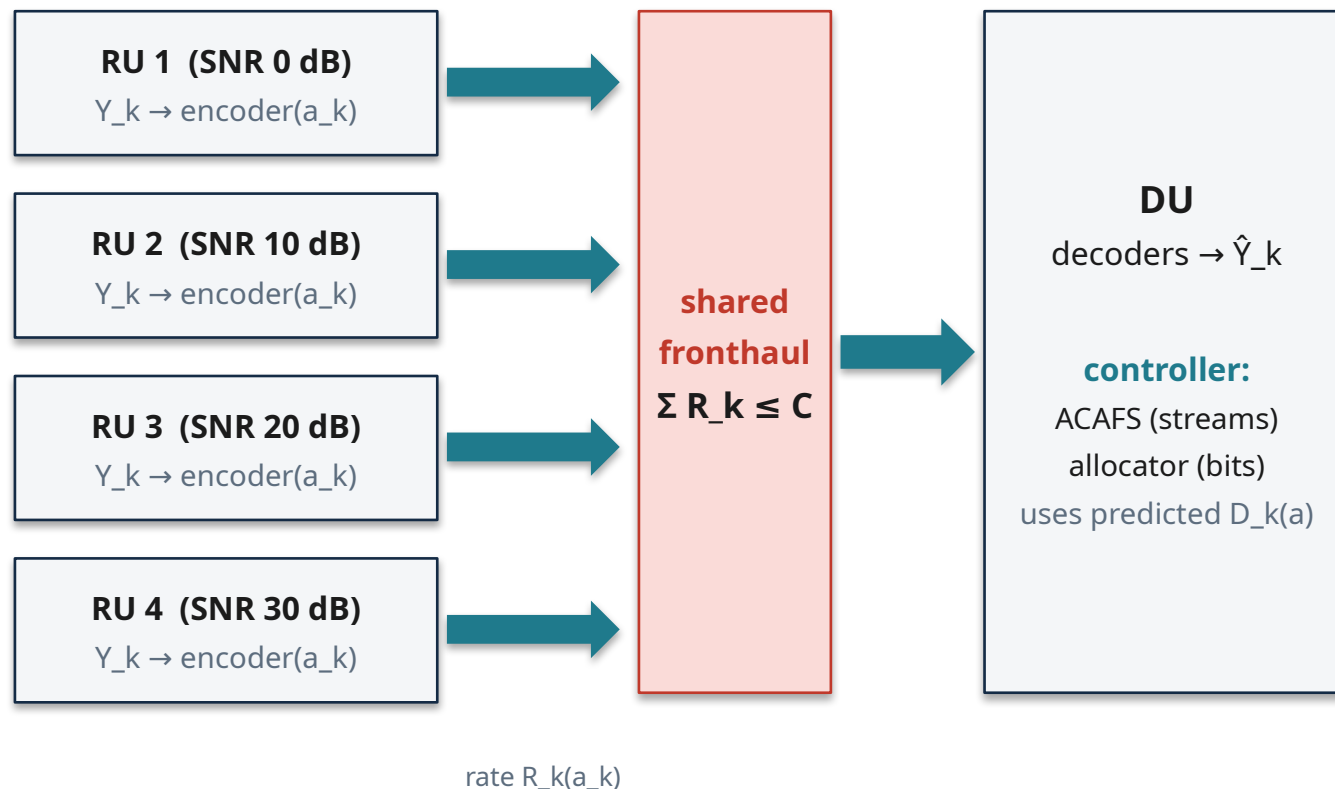
Research context and the gap this project addresses

Verified sources only; full list with links in docs/research_review.md (search cutoff 17 Sep 2026)

Direction	Representative work	What it assumes / measures	Relation to this project
Standardised I/Q compression	O-RAN WG4 CUS Annex A [5]; Silva et al., JCIS 2022 [6]	per-PRB BFP / block scaling / μ -law, 4-bit exponent; SQNR vs iqWidth, single RU	our BFP baseline reproduces this bit accounting; their per-PRB method selection $\approx$ our per-RU menu, but no capacity constraint
Fronthaul survey / functional splits	Park et al., IEEE SPM 2014 [1]; Larsen et al., COMST 2019 [3]; 3GPP TR 38.801 Annex A [4]	quantize-and-forward theory; UL bit-rate per split: Opt.6 $\approx$ 5.6 Gb/s $\ll$ Opt.7b 54-86 $\ll$ Opt.8 157 Gb/s	source of the split-ordering correction in ACAFS; motivates rank-adaptive stream count
Shared fronthaul, centralised control	Lagén et al., IEEE ComMag 2021/2022 [7][9]; IEEE TMC 2023 [8]	several cells on one FH link; DL modulation compression + MAC scheduling; ns-3 throughput	template for Exp5 (per-cell operating point, common capacity); ours is UL I/Q with NMSE, no MAC
Structured compression of channel / received matrices	Wen et al., WCL 2018 (CsiNet) [15]; Qiao-Jiang-Yu, ICC 2023 / TWC 2024 [13][14]; Rahmani et al., arXiv 2025 [10]	delay-domain sparsity of H; learned dimension-reduction matrices from local CSI; O-RAN BFP under TDL, BLER	explains why CSEE works only on reference symbols; low-rank sketch = our RAS-BFP; BLER is the metric we still lack
Closest single-link prior art	Aswathylakshmi-Ganti, WCNC 2022 [22]; Li et al., IEEE Access 2019 [24]; Kanno et al., VTC 2023 [25]	64-ant. TDL-A delay-domain low rank via blind deconvolution; bit-width vs load/SINR on one eCPRI link; SVD streams + bits per stream at one AP	CSEE's structure, but data symbols removed first — hence CSEE needs reference symbols; allocator extends [24][25] to many RUs
Learning-based split orchestration	Murti et al., EuCNC 2022 [16]; IEEE TNSM 2024 [17]	DQN / D3QN over minutes-scale demand traces; cost model from testbed; up to 59-69 % cost saving vs static	justifies adaptive split as a topic; RL not warranted here because our rate model is closed-form per symbol
Numerical foundations	Halko-Martinsson-Tropp, SIAM Rev. 2011 [19]; Tropp 2011 [20]; Shoham-Gersho 1988 [21]	randomised range finder + Frobenius bound; SRHT subspace embedding; λ -sweep / convex-hull bit allocation	RAS-BFP algorithm and Thm-2 bound; allocator = Shoham-Gersho on RU menus (no algorithmic novelty claimed)

Gap addressed: prior art adapts bits or streams on one link [22][24][25]; here a closed-form per-symbol predictor drives allocation across RUs sharing one link, with a metric separating signal loss from removed noise.

3GPP TR 38.901 TDL-A uplink, one OFDM symbol per RU



$$Y_k = H_k \cdot \text{diag}(x_k) + W_k \in \mathbb{C}^{M \times N}$$

$H_k = \sum_l a_l e_l^T$: 23 TDL-A taps at delays τ_l (100 ns RMS),
 $a_l \sim \text{CN}(0, P_l R)$, $R_{ij} = 0.7^{|i-j|}$

$\Rightarrow \text{rank}(H) \leq 23$ (spatial low rank) and IFFT over subcarriers
 is concentrated on few delay bins

x_k : data symbols (random QPSK per subcarrier) or
 reference symbols (known, de-rotated $\rightarrow H + W'$)

$W \sim \text{CN}(0, \sigma^2)$, $\sigma^2 = 10^{(-\text{SNR}/10)}$, $E|H|^2 = 1$

Metrics the model can measure honestly

NMSE (dB) = $10 \log_{10} \|Y - \hat{Y}\|_F^2 / \|Y\|_F^2$ — vs transported Y ,
 and vs noiseless H

Compression ratio = $2 \cdot M \cdot N \cdot 16$ / bits used (exact accounting
 incl. exponents, headers)

Encode time: numpy medians — relative comparison only

Allocation: utilisation, dropped cells, |predicted –
 measured| NMSE, decision time

Formulation and the predictor that makes it cheap

[new]

Multiple-choice knapsack per scheduling interval

Decision

$a_k \in \text{menu}$ (e.g. CSEE (K, b), BFP b) for each RU k

Rate $R_k(a)$: exact bit count · Distortion $D_k(a) = \|Y_k - \hat{Y}_k\|^2 / \|Y_k\|^2$

minimise $\sum_k D_k(a_k)$ [or $\max_k D_k(a_k)$]
subject to $\sum_k R_k(a_k) \leq C_{\text{link}}$

Solver: greedy marginal gain on each RU's lower convex hull of (R, D) points — Lagrangian optimum up to one fractional step; overload → drop least-useful RUs.

Measured: ≈ 1 ms allocation + ≈ 27 ms predictions for 8 RUs × 66 options (Python).

Proposition 1 (CSEE rate-distortion predictor)

$Y_d = \text{IFFT}_N(Y)$ row-wise; $S = \text{top-}K \text{ delay bins by } \sum_m |Y_d[m,n]|^2$

$$\text{NMSE} = (T_S + E_q) / \|Y_d\|_F^2$$

$T_S = \sum_{n \notin S} \|Y_d[:,n]\|^2$ — discarded tail, exact (Parseval)

$E_q = \sum_{\text{blocks}} n_b \Delta_b^2 / 6$ (estimate) or $n_b \Delta_b^2 / 2$ (worst case, true bound), $\Delta_b = 2^{\{e_b\}}$ from BFP block exponents

Noise-aware form: subtract the noise expected in the tail, charge noise in kept bins, divide by signal energy — uses the RU's σ^2 estimate

Cost: one IFFT per RU for the whole menu (nested top-K supports).

Measured |predicted – measured| ≤ 0.5 dB for CSEE in Exp5 (BFP predictor: ≤ 1.3 dB).

Theorem 2 (RAS-BFP reference bound, from HMT 2011 Thm 10.5, Gaussian sketch, $p \geq 2$)

$$E\|Y - \hat{Y}_r\|_F^2 \leq (4 + 2r/(p-1)) \cdot \sum_{\{i > r\}} \sigma_i^2$$

Encoders: two baselines, two proposed

[new]

All quantise with O-RAN-style block floating point (12-RE blocks, 4-bit exponent)

Encoder	Structure exploited	Payload	Complexity	Analysis
BFP (O-RAN WG4)	dynamic range only	$2bMN + 4 \cdot M \cdot N / 12$	$O(MN)$	quantisation-noise model
Truncated SVD	spatial low rank	$BFP(U_r \Sigma_r) + BFP(V_r)$	$O(MN \cdot \min(M, N))$ (economy)	Eckart-Young floor
CSEE (proposed)	delay-domain sparsity, shared support across antennas	$BFP(Y_d[:, S]) + K \cdot \lceil \log_2 N \rceil$	$O(MN \log N)$	Prop. 1 (exact tail)
RAS-BFP (proposed)	spatial low rank via randomised range finder: SRHT sketch $\rightarrow QR \rightarrow YQ \rightarrow$ small SVD	same shape as SVD at rank r	$O(MN(r+p))$	Thm. 2 (HMT-type)

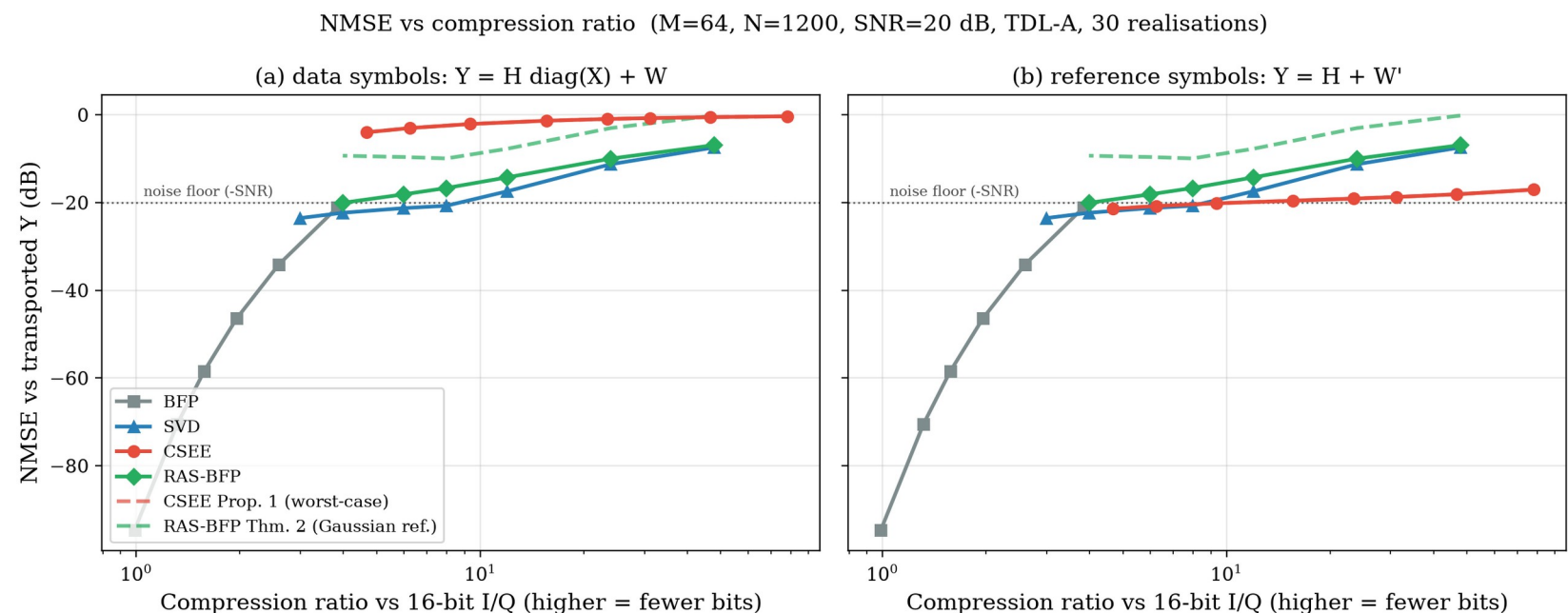
• Which structure survives modulation? $Y = H \cdot \text{diag}(x) + W$

- Spatial rank: $\text{rank}(H \cdot \text{diag}(x)) = \text{rank}(H)$ because $\text{diag}(x)$ is invertible $\rightarrow$ SVD and RAS-BFP work on data and reference symbols.
- Delay sparsity: $\text{IFFT}(H \cdot \text{diag}(x)) = \text{IFFT}(H) \otimes \text{IFFT}(x)$; random QPSK x is white $\rightarrow$ the convolution spreads energy over all N bins $\rightarrow$ CSEE only works on reference symbols ($H + W'$).
- This was not visible in the original documents because the original simulation used an unmodulated (constant) symbol.

Result 1 — rate-distortion frontier (Exp1)

[new]

$M=64, N=1200, SNR\ 20\ dB, 30\ channel\ realisations; bits\ b=10\ for\ SVD/CSEE/RAS-BFP$



Operating point	CR	data	reference
BFP $b=8$	2.0x	-46.4 dB	-46.4 dB
SVD $r=12$	8.0x	-20.7 dB	-20.7 dB
RAS-BFP $r=12$	8.0x	-16.7 dB	-16.7 dB
CSEE $K=120$	15.6x	-1.3 dB	-19.6 dB

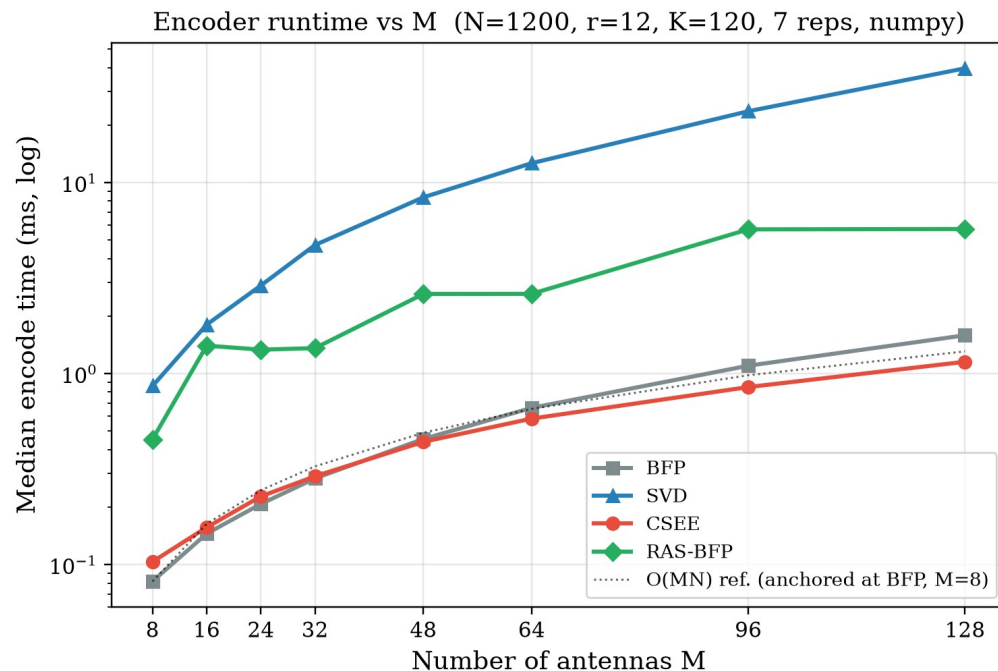
- Workload: TDL-A, SNR 20 dB; metric: NMSE vs transported Y
- CSEE reaches -17 to -21 dB at 6–78x CR on reference symbols but ≈ 0 dB on data symbols.
- RAS-BFP is 1–4 dB worse than SVD at equal payload; Thm-2 bound 6–10 dB above (valid, loose).
- All truncating encoders sit at the -20 dB noise floor of this metric at 20 dB SNR.

CSEE is a reference-symbol (CSI) compressor, not a general I/Q compressor; spatial low-rank methods work on both symbol types.

Results 2-3 — encode cost and two NMSE references

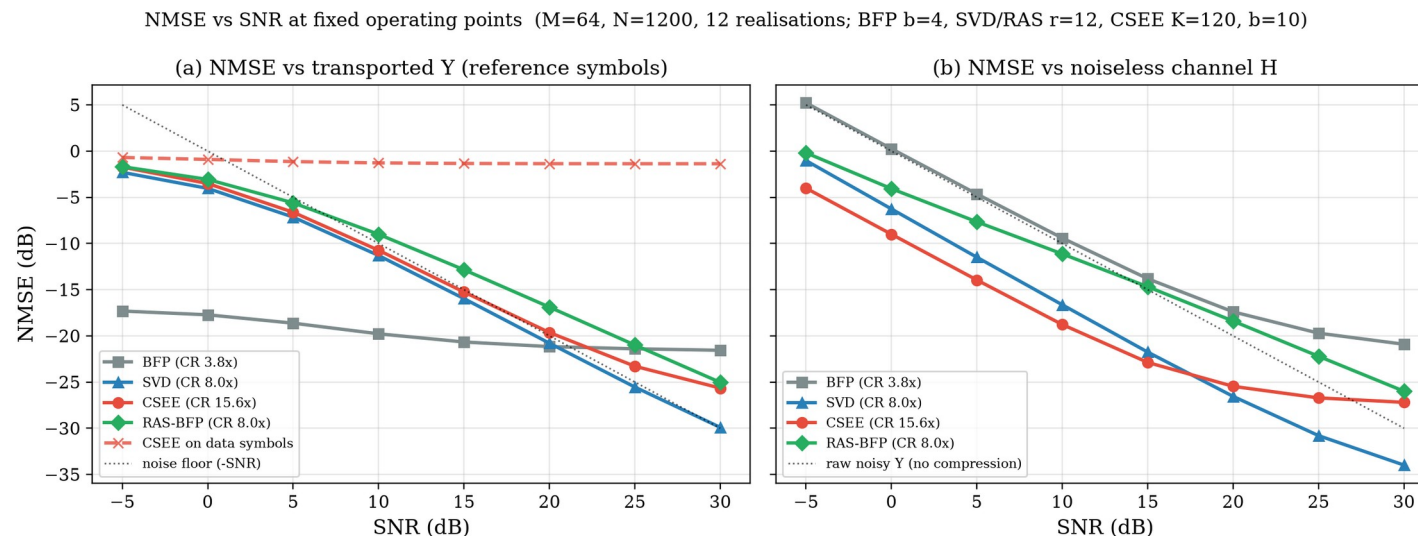
[new]

Exp2: numpy medians of 7, $N=1200$ · Exp4: reference symbols, $CR \approx 8x$ operating points, 12 realisations



M=64: BFP 0.66 ms · CSEE 0.58 ms · RAS-BFP 2.6 ms · SVD 12.6 ms

RAS-BFP $\approx 5x$ faster than economy SVD at $M=64$, $7x$ at $M=128$ (was reported as $\sim 100x$ with a full SVD)



(a) vs Y: truncating encoders track $-\text{SNR}$. (b) vs H: at 0 dB SNR CSEE gives -9 dB and SVD -6 dB where the raw noisy matrix is at 0 dB — they denoise.

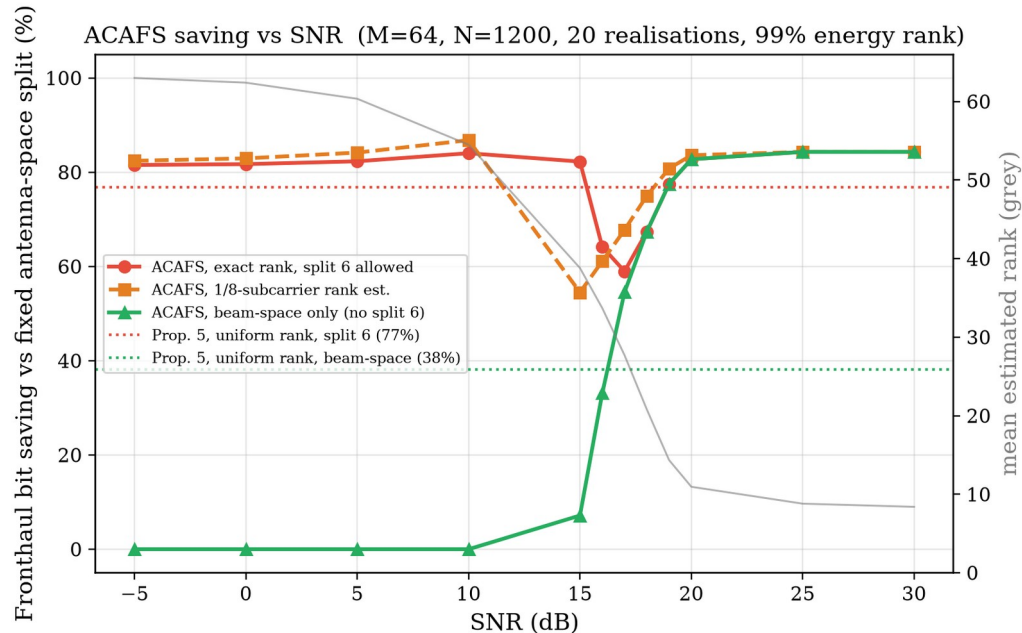
At 30 dB: SVD -34, CSEE -27, RAS-BFP -26, BFP $b=4$ -21 dB. CSEE on data symbols (dashed): ≈ -1 dB everywhere.

The earlier claim 'stable across SNR' was an artefact of the metric; against the noiseless channel the encoders act as denoisers at low SNR.

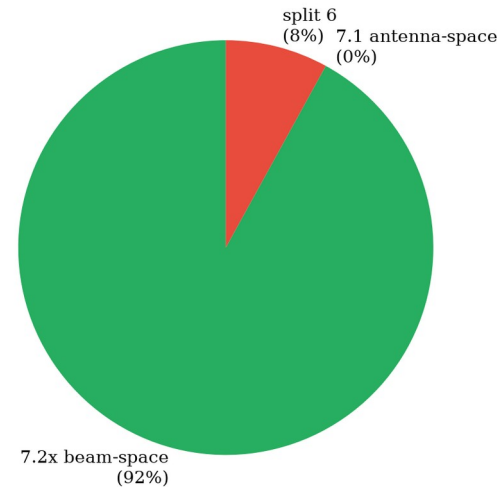
Result 4 — rank-adaptive split selection (ACAFS, Exp3)

[fix]

$M=64$, $N=1200$, 20 realisations per SNR; rank = 99 % energy of $Y Y^H$ eigenvalues



Split decisions, SNR 16-20 dB



• Corrected bandwidth model (3GPP TR 38.801 / O-RAN)

- antenna-space 7.1/7.2x-A: 2MNb (reference)
- beam-space 7.2x-B: 2rNb + weights/14
- split 6 (MAC-PHY): $r \cdot N \cdot \eta$, $\eta = 6$ bit/RE $\rightarrow \approx 20x$ cheaper, needs full UL PHY in the RU
- Rule: $r/M < \tau_{\text{high}} \rightarrow$ beam-space with $\max(r, [\tau_{\text{low}} M])$ streams; else split 6 if RU compute allows, else antenna-space.

• Beam-space only: 0 % below 15 dB $\rightarrow$ 83 % at 20 dB (mean rank 11).

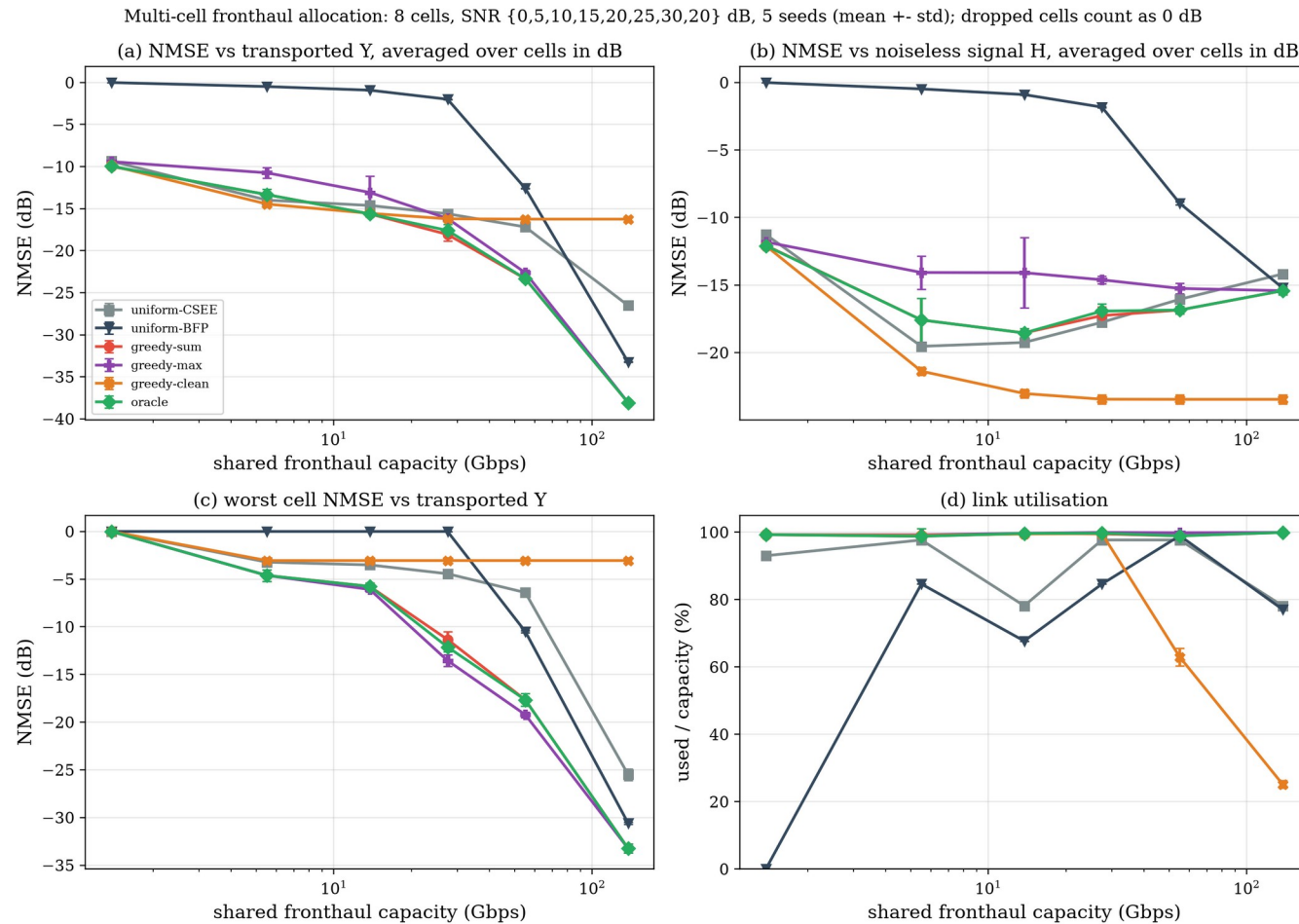
- Fast estimator (1/8 subcarriers): 36 % less time, but under-estimates rank by up to 10 at 10–17 dB $\rightarrow \tau_{\text{low}}$ floor is the guard.
- Prop. 5 (uniform rank): 38 % beam-space, 77 % with split 6 — a prediction, not a bound.

The original 'saving vs split 6' figure had the split ordering inverted; the corrected model shows saving is driven by rank collapse at high SNR and by whether the RU may host split 6.

Result 5 — shared-capacity allocation, 8 RUs (Exp5)

[new]

SNR per RU {0,5,10,15,20,25,30,20} dB · menu 11 $K \times 6 b = 66$ CSEE points · 5 seeds · reference symbols



• Policies

- uniform-CSEE / uniform-BFP: one operating point for all RUs (today's static config)
- greedy-sum / greedy-max: proposed, decisions from Prop. 1 predictions only
- greedy-clean: proposed, noise-aware objective (signal distortion)
- oracle: greedy on measured NMSE — encodes every option (ablation)

• What the panels show

- (a) vs Y: greedy within 0.5 dB of the oracle at every capacity; 2.5 dB better than uniform at 27.5 Gbps, 6.2 dB at 55 Gbps; uniform-BFP cannot fit below 55 Gbps (drops RUs)
- (b) vs H: greedy-sum wastes bits on noise in low-SNR RUs; noise-aware objective is 6.2 dB better at 27.5 Gbps
- (c) min-max objective: worst RU up to 2.2 dB better (27.5 Gbps) at a 0.7–2.6 dB cost in the mean
- (d) noise-aware uses only 25 % at 138 Gbps — more bits stop helping the signal

Prediction-driven allocation matches an encode-everything oracle; the objective matters — optimise signal distortion, not fidelity to noise.

Supporting result — compute cost of a split decision

[repo]

MMSE-type processing needs matrix inverses per PRB group; timings on a 4-core CPU, float32, held-out matrices

n	LAPACK LU (numpy.linalg.inv)	classic Newton–Schulz	InverseNet-NS (17 params, learned)	InverseNet-Ultra (learned)	InverseNet-MLP
10	0.00581 ms · err 4.8e-08	0.0423 ms · err 9.3e-08	0.114 ms · err 4.0e-02	0.0383 ms · err 1.7e-01	0.0501 ms · err 3.6e-01
100	0.138 ms · err 4.8e-08	0.403 ms · err 2.2e-07	0.328 ms · err 4.0e-04	0.397 ms · err 1.1e-06	n/a
500	5.86 ms · err 4.8e-08	18.1 ms · err 3.6e-07	117 ms · err 7.4e-03	17.9 ms · err 1.3e-06	n/a

- LAPACK LU is fastest and float32-exact at every size; a 100×100 inverse (≈ 0.14 ms) fits a 500 μ s slot per PRB group on one core, a 500×500 one (≈ 6 ms) does not.
- Learned unrolled Newton–Schulz (17 parameters) generalises to 4e-4 error at n=100 (4e-2 at n=10, 7e-3 at n=500) but is slower than LU; direct-regression MLP fails (0.36 error at n=10). Ultra reaches ≈ 1 e-6 at n=100/500 but 0.17 at n=10 and never beats LU medians.
- Fixed this iteration: the NumPy inference engine used ReLU while the trained model uses GELU (now identical, tested); overstated docstring corrected; benchmark re-run (n=500 with 20 held-out matrices).
- Role in the project: an RU-compute constraint for the split-6 option in ACAFS — connecting the two halves is [future] work.

Contributions, limitations and plan to final evaluation

• Completed

- [new] • Runnable, tested simulator for channel → encoders → split → allocation (was missing)
- [fix] • Corrected baselines and models (economy SVD, split ordering, real rank estimator, two NMSE references)
- [new] • Prop. 1 predictor within 0.5 dB; Thm. 2 reference bound checked
- [new] • Multi-cell allocator with static baselines, oracle and objective ablations, 5 seeds
- [new] • Finding: delay-domain compression is CSI-only; spatial low rank is modulation-invariant
- [repo] • Compute benchmark, bug-fixed and re-run

• Limitations

- Single symbol; no scheduler, HARQ, latency, multi-slot dynamics
- CSEE valid on reference symbols only → Exp5 covers reference-symbol traffic
- Python timings: ordering and scaling only
- Thm. 2 proven for Gaussian sketch; SRHT checked empirically
- Split-6 rate model coarse ($\eta = 6$ bit/RE); σ^2 assumed known in the noise-aware ablation
- No real traces; only TDL-A
- *No novelty claimed for any single encoder; the contribution is the validated system and its findings*

• Next (to final evaluation)

- [future] • Slot-level simulation mixing data + reference symbols; mixed CSEE/RAS-BFP/BFP menu (needs RAS-BFP predictor)
- [future] • Joint ACAFS + allocator under one capacity with the compute benchmark as RU constraint
- [future] • Sensitivity to σ^2 estimation error and rank bias
- [future] • Learned predictor only where no analytical one exists, judged against the analytical one
- [future] • C/SIMD timing of BFP, CSEE, RAS-BFP

References (1/2) — verified sources, see docs/research_review.md

- [1] S.-H. Park, O. Simeone, O. Sahin, S. Shamai, "Fronthaul compression for cloud radio access networks," IEEE Signal Process. Mag., vol. 31, no. 6, 2014. doi:10.1109/MSP.2014.2330031 (abstract)
- [2] M. Peng, C. Wang, V. Lau, H. V. Poor, "Fronthaul-constrained cloud radio access networks: Insights and challenges," IEEE Wireless Commun., vol. 22, no. 2, 2015. (full text, author copy)
- [3] L. M. P. Larsen, A. Checko, H. L. Christiansen, "A survey of the functional splits proposed for 5G mobile crosshaul networks," IEEE Commun. Surveys Tuts., vol. 21, no. 1, 2019. (full text)
- [4] 3GPP TR 38.801 (Rel-14), "Study on new radio access technology: Radio access architecture and interfaces," §11.1.2, Annex A Tables A-1/A-2.
- [5] O-RAN Alliance WG4, "Control, User and Synchronization Plane Specification," O-RAN.WG4.TS.CUS.0-R004-v19.00, Annex A (compression methods). (annex listing; BFP details cross-checked with [6])
- [6] M. D. L. Silva, L. Ramalho, I. Almeida, E. Medeiros, M. Berg, A. Klautau, "A new O-RAN compression approach for improved performance on uplink signals," J. Commun. Inf. Syst., vol. 37, no. 1, pp. 30–41, 2022. doi:10.14209/jcis.2022.3 (full text)
- [7] S. Lagén, L. Giupponi, A. Hansson, X. Gelabert, "Modulation compression in next generation RAN: Air interface and fronthaul trade-offs," IEEE Commun. Mag., vol. 59, no. 1, 2021. doi:10.1109/MCOM.001.2000453 (full text)
- [8] S. Lagén, X. Gelabert, L. Giupponi, A. Hansson, "Fronthaul-aware scheduling strategies for dynamic modulation compression in next generation RANs," IEEE Trans. Mobile Comput., vol. 22, no. 5, pp. 2725–2740, 2023. (abstract)
- [9] S. Lagén, X. Gelabert, A. Hansson, M. Requena, L. Giupponi, "Fronthaul compression control for shared fronthaul access networks," IEEE Commun. Mag., vol. 60, no. 6, 2022. doi:10.1109/MCOM.001.2100959 (full text)
- [10] M. Rahmani et al., "Exploring O-RAN compression techniques in decentralized distributed MIMO systems: Reducing fronthaul load," arXiv:2507.04997, 2025. (preprint, full text)
- [11] "Hardware implementation for O-RAN block floating point compression and decompression methods in 5G base station," Proc. ATC 2024. doi:10.1109/ATC63255.2024.10908174 (abstract)
- [12] S. Yagi et al., "Deep learning-based nonlinear quantizer for fronthaul compression," Proc. OECC/PSC 2022. (abstract)
- [13] R. Qiao, T. Jiang, W. Yu, "Learning-based fronthaul compression for uplink cloud radio access networks," Proc. IEEE ICC 2023, pp. 5928–5933. (abstract)
- [14] R. Qiao, T. Jiang, W. Yu, "Meta-learning-based fronthaul compression for cloud radio access networks," arXiv:2403.09004, 2024; IEEE Trans. Wireless Commun., vol. 23, no. 9, 2024. (abstract)
- [15] C.-K. Wen, W.-T. Shih, S. Jin, "Deep learning for massive MIMO CSI feedback," IEEE Wireless Commun. Lett., vol. 7, no. 5, pp. 748–751, 2018. (full text)

References (2/2) — verified sources, see docs/research_review.md

- [16] F. W. Murti, S. Ali, G. Iosifidis, M. Latva-aho, "Learning-based orchestration for dynamic functional split and resource allocation in vRANs," Proc. EuCNC/6G Summit 2022. (full text)
- [17] F. W. Murti, S. Ali, G. Iosifidis, M. Latva-aho, "Deep reinforcement learning for orchestrating cost-aware reconfigurations of vRANs," IEEE Trans. Netw. Service Manag., vol. 21, no. 1, pp. 200–216, 2024. doi:10.1109/TNSM.2023.3292713 (full text)
- [18] F. W. Murti, S. Ali, M. Latva-aho, "Constrained deep reinforcement based functional split optimization in virtualized RANs," arXiv:2106.00011, 2021. (abstract)
- [19] N. Halko, P.-G. Martinsson, J. A. Tropp, "Finding structure with randomness," SIAM Rev., vol. 53, no. 2, pp. 217–288, 2011. doi:10.1137/090771806 (full text)
- [20] J. A. Tropp, "Improved analysis of the subsampled randomized Hadamard transform," Adv. Adapt. Data Anal., vol. 3, no. 1–2, pp. 115–126, 2011. (full text, preprint)
- [21] Y. Shoham, A. Gersho, "Efficient bit allocation for an arbitrary set of quantizers," IEEE Trans. Acoust., Speech, Signal Process., vol. 36, no. 9, pp. 1445–1453, 1988. doi:10.1109/29.90373 (abstract)
- [22] Aswathylakshmi P, R. K. Ganti, "Fronthaul compression for uplink massive MIMO using matrix decomposition," Proc. IEEE WCNC 2022, pp. 2524–2529. doi:10.1109/WCNC51071.2022.9771783 (full text)
- [23] F. Wiffen, W. H. Chin, A. Doufexi, "Distributed dimension reduction for distributed massive MIMO C-RAN with finite fronthaul capacity," Proc. Asilomar 2021, pp. 1228–1236. doi:10.1109/IEEECONF53345.2021.9723180 (full text)
- [24] L. Li et al., "Enabling flexible link capacity for eCPRI-based fronthaul with load-adaptive quantization resolution," IEEE Access, vol. 7, pp. 102174–102185, 2019. doi:10.1109/ACCESS.2019.2930214 (full text)
- [25] I. Kanno et al., "Adaptive bit allocation for SVD based hybrid processing of uplink cell-free massive MIMO under limited fronthaul capacity," Proc. IEEE VTC2023-Spring. doi:10.1109/VTC2023-Spring57618.2023.10201009 (abstract)
- [26] L. Liu, R. Zhang, "Optimized uplink transmission in multi-antenna C-RAN with spatial compression and forward," IEEE Trans. Signal Process., vol. 63, no. 19, pp. 5083–5095, 2015. (abstract)
- [27] A. Martínez Alba, W. Kellerer, "Dynamic functional split adaptation in next-generation radio access networks," IEEE Trans. Netw. Service Manag., vol. 19, no. 3, pp. 3239–3263, 2022. (abstract)
- [28] L. Wang, S. Zhou, "On the fronthaul statistical multiplexing gain," IEEE Commun. Lett., vol. 21, no. 5, pp. 1099–1102, 2017. (abstract)
- [29] 3GPP TR 38.901 V19.4.0, "Study on channel model for frequencies from 0.5 to 100 GHz," cl. 7.7.2–7.7.5 (TDL-A, delay scaling, correlation extension).

Backup — audit: verified problems in original material

- src/ absent from every commit and branch; Exp1-4 + notebook import it → nothing ran
- Exp2 SVD baseline: committed figure ~ 1.7 s at $M=64$ vs 13 ms economy SVD; $O(MN)$ reference line plotted in seconds on a ms axis
- Exp3: 'fast rank est.' = exact rank + Uniform $\{-1,0,1\}$; 'Theorem 5 bound' below the simulated curve → not a bound
- ACAFS: split 6 treated as the most expensive split; TR 38.801 ordering is $R_6 \ll R_{7.2x\text{-beam}} \leq R_{7.x\text{-antenna}}$
- Exp4: 'stable across SNR' claimed while the figure shows $NMSE \approx -SNR$
- CSEE input: constant pilot $\Rightarrow$ delay sparsity; with random QPSK per subcarrier CSEE gives ≈ -0.5 dB
- CR reference 32-bit components $\Rightarrow$ all CRs inflated 2x (BFP b=8: 3.9x quoted vs 2.0x against 16-bit)
- Docs: 'Theorems 1-5' without statements/proofs; '100 MHz at 15 kHz' with $N=1200$ (=18 MHz); TWC/JSAC target; AVX-512/CUDA/USRP roadmap
- matinv_bench: ReLU vs GELU in the NumPy engine; docstring claiming $1e-7$ error and LU-beating latency
- Original figures/tables preserved in results/original_snapshot/ for comparison

Backup — Proposition 1 and the noise-aware variant

- Setup: $Y \in \mathbb{C}^{M \times N}$; $Y_d = \text{IFFT}_N(Y)$ row-wise (numpy: $Y = \text{FFT}(Y_d)$, $\|Y\|_F^2 = N\|Y_d\|_F^2$). Encoder keeps columns S ($|S|=K$) and BFP-quantises them: $\hat{Y}_d = Q(Y_d[:,S])$ on S , 0 elsewhere. Decoder: $\hat{Y} = \text{FFT}(\hat{Y}_d)$.
- $\|Y - \hat{Y}\|_F^2 = N\|Y_d - \hat{Y}_d\|_F^2 = N(\sum_{n \notin S} \|Y_d[:,n]\|^2 + \|Y_d[:,S] - Q(Y_d[:,S])\|_F^2) = N(T_S + E_q)$.
- $\text{NMSE} = N(T_S + E_q) / (N\|Y_d\|_F^2) = (T_S + E_q) / \|Y_d\|_F^2$. T_S is exact; E_q depends on the quantiser.
- BFP block with exponent e_b , step $\Delta_b = 2^{e_b}$: per real component $|\text{error}| \leq \Delta_b/2$ (deterministic) and $\text{Var} \approx \Delta_b^2/12$ (high-rate model). Complex sample: $\leq \Delta_b^2/2$ worst case, $\approx \Delta_b^2/6$ expected. Summing over the n_b non-zero samples of each block gives E_q^{wc} (bound) and E_q^{est} (estimate). Zero samples quantise exactly.
- Top- K supports are nested in K , so one sort of the N bin energies gives T_S for all K ; block maxima per K give Δ_b for all b as a shift of $\log_2(2^{b-1}-1)$. Whole 66-point menu $\approx$ one encoder pass.
- Noise-aware form (RU knows σ^2): white noise has total delay-domain energy $M\sigma^2$ spread evenly over N bins. Signal distortion $\approx [\max(T_S - M\sigma^2(N-K)/N, 0) + M\sigma^2 K/N + E_q] / (\|Y_d\|_F^2 - M\sigma^2)$. Measured within 0.1 dB at 20 dB SNR and ≈ 1.7 dB at 0 dB SNR.
- Why it matters: with the plain form the allocator upgrades low-SNR RUs to reproduce noise; with the noise-aware form the marginal gain of extra bits goes to zero once the tail is noise, so capacity is released.

Backup — RAS-BFP algorithm and Theorem 2

- 1. Sketch antennas: $Z = \Omega Y \in \mathbb{C}^{(r+p) \times N}$, $\Omega = \sqrt{M/(r+p)} \cdot S \cdot H_M \cdot D$ (SRHT: random signs D , Walsh–Hadamard H_M via fast transform, row sampler S) or Gaussian.
- 2. Thin QR: $Z^H = Q R$, $Q \in \mathbb{C}^{N \times (r+p)}$ spans an approximate dominant right singular subspace of Y .
- 3. Project: $L = Y Q$ ($M \times (r+p)$); small SVD of L ; keep rank r : $L_r = U_r \Sigma_r$, $Q_r = Q V_r$. $\hat{Y} = L_r Q_r^H$ (HMT Alg. 4.1 + 5.1).
- 4. Transmit BFP(L_r) and BFP(Q_r) — identical payload shape to truncated SVD at rank $r \Rightarrow$ equal CR, fair comparison.
- Cost: $O(MN \log M)$ sketch (SRHT) or $O(MN(r+p))$ (Gaussian) + $O(N(r+p)^2)$ QR + $O(MN(r+p))$ projection, vs $O(MN \cdot \min(M, N))$ for the SVD.
- Theorem 2 (HMT 2011, Thm 10.5, Gaussian Ω , $p \geq 2$): $E \|Y - Y Q Q^H\|_F^2 \leq (1 + r/(p-1)) \sum_{i>r} \sigma_i^2$. Truncation adds at most an Eckart–Young term ($\|Y - Q[Q^H Y]_r\| \leq \|Y - Q Q^H Y\| + \|Y - Y_r\|$), so $E \|Y - \hat{Y}\|_F^2 \leq (4 + 2r/(p-1)) \sum_{i>r} \sigma_i^2$. Quantisation is on top.
- Caveats: proven for Gaussian sketches; SRHT (Tropp 2011) needs larger oversampling for a proof — with $p = 4$ the bound holds empirically with 6–10 dB slack in Exp1. For $p < 2$ the code returns ∞ rather than a made-up bound. SRHT falls back to Gaussian when M is not a power of two or $r+p \geq M$.

Backup — ACAFS bandwidth model and allocator

- **Bits per OFDM symbol per RU ($M=64$, $N=1200$, $b=16$, $\eta=6$, $T_c=14$)**

- antenna-space: $2MNb = 2\,457\,600$
- beam-space, r streams: $2rNb + 2Mrb/T_c$ ($r=10$: 385 463; $r=32$: 1 233 481)
- split 6, r layers: $rN\eta$ ($r=10$: 72 000; $r=64$: 460 800)
- rule: $\rho = r/M$; $\rho < \tau_{\text{high}}=0.55 \rightarrow$ beam-space with $\max(r, \lceil 0.15 \cdot M \rceil=10)$ streams; else split 6 if allowed, else antenna-space
- Prop. 5 = mean over $r \in \{1..M\}$ of the per-rank saving under the rule (a prediction; simulated saving can exceed it when ranks concentrate low)

- **greedy_allocate(rates, dist, C, objective)**

- for each RU: keep only (R, D) points on the lower convex hull (dominated / non-convex points never help)
- start every RU at its cheapest point; if $\sum R > C$: drop RUs with the largest predicted D at that point until it fits (they gain least from transport)
- repeat: among feasible next-hull upgrades pick $\arg\max \Delta D / \Delta R$ (sum) or the RU with current max D (max); stop when none fits
- guarantee: Lagrangian-relaxation optimum up to one fractional upgrade (standard for multiple-choice knapsack greedy on convex hulls)
- uniform_allocate: best single point that fits for all RUs — the static-configuration baseline
- tests: capacity respected, greedy $\geq$ uniform on its objective, low load $\rightarrow$ best point, overload $\rightarrow$ drops, zero-traffic RU, invalid inputs

Backup — reproducibility: commands and outputs

- `pip install -r requirements.txt` (numpy, scipy, pandas, matplotlib, tqdm, pytest; torch only for `matinv_bench`)
- `python -m pytest tests -q` → 42 passed (~1 s)
- `python experiments/exp1_nmse_vs_cr.py` → `results/figures/exp1_*.png | pdf, results/data/exp1_nmse_vs_cr.json` (~10 s)
- `python experiments/exp2_complexity.py` → `exp2_*` (~2 s)
- `python experiments/exp3_acafs_gain.py` → `exp3_*` (~3 s)
- `python experiments/exp4_snr_sweep.py` → `exp4_*` (~5 s)
- `python experiments/exp5_fronthaul_allocation.py` → `exp5_allocation.png, exp5_allocation_runs.csv` (every run), `exp5_allocation_summary.csv` (~8 s)
- `jupyter nbconvert --execute --inplace notebooks/mid_evaluation_demo.ipynb` (from `notebooks/`)
- `python -m matinv_bench.generate_dataset --dims 10 100 --num-samples 1000`; `python -m matinv_bench.benchmark --dims 10 100`
- Every `results/data` file records generation time, Python/numpy version, host and CPU count; `results/logs/` has the console output of the runs shown here.
- Host used for the numbers in this deck: 4-core x86_64, Python 3.12, numpy 2.x, no GPU.